



ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled "**EXAM MANAGEMENT SYSTEM**" is submitted by

PRANAV VEDULA

(24B11CS355)

**MEDAPATI SATYA HARSHA
VARDHAN REDDY**

(24B11ICS275)

POSA KAVERI

(24B11CS352)

BODHANPU SUHAS

(24B11CS049)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Information Technology for the academic year 2025-2026.

Project Guide

S.Saikala
Assistant Professor,
Department of CSE

Head of the Department

Dr. T Sudha Rani, M.Tech., Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**EXAM MANAGEMENT SYSTEM**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree in Information Technology**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

PRANAV VEDULA	(24B11CS355)
M SATYA HARSHA	(24B11CS275)
VARDAHAN REDDY	
POSA KAVERI	(24B11CS352)
BODHANAPU SUHAS	(24B11CS049)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Ms. S.Saikala, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr.K.Chandra Sekhar, Associate Professor and Dr.Hari Jyothula, Associate Professor** Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. T Sudha Rani, Associate Professor**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr S.Rama S r e e , Pro Vice-Chancellor (Academics), D r. M.B. Srinivas, V i c e -Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of contents

CONTENTS	PAGE NO.
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	5
2.3.1 Minimum Hardware Requirements	5

2.3.2 Software Requirements	5
3. TECHNOLOGY DESCRIPTION	6
3.1 Programming Language	6
3.1.1 Introduction to Java	6
3.2 Framework / Libraries	6
3.2.1 Introduction to Java Swing	6
3.3 Database Technology	7
3.3.1 JDBC (Java Database Connectivity)	7
3.3.2 MySQL Database	7
3.4 Tools & IDEs	8
3.4.1 Visual Studio Code / IntelliJ IDEA	8
4. DATABASE DESIGN	9
4.1 Database Tables	9
4.2 Table Structure	9
4.3 Keys and Constraints	11
4.4 Table Relationships	12

5. SYSTEM DESIGN	13
5.1 System Architecture	13
5.2 Modules Description	13
6. SYSTEM IMPLEMENTATION	15
7. CONCLUSION	20
8. FUTURE ENHANCEMENTS	21
9. REFERENCES	22
APPENDIX (Sample Code)	23
APPENDIX (SQL Queries)	25

ABSTRACT

The Exam Seating System is a desktop-based application developed to automate and streamline the process of allocating examination seating for students in a university environment. The system replaces traditional manual seating arrangements with a comprehensive digital solution that manages exam rooms, student data, faculty invigilators, timetables, and hall ticket generation in a centralized manner.

Developed using Core Java for application logic, Java Swing for the graphical user interface, JDBC for database connectivity, and MySQL as the backend database, the application provides a role-based login system supporting two roles: Administrator and Examiner. The Administrator manages room configurations, student data import, user accounts, and backup operations. The Examiner handles seating schedule generation, timetable management, hall ticket printing, invigilator roster management, and exam attendance tracking.

The system implements a column-wise seat-filling algorithm that distributes students across available exam halls while skipping marked broken or damaged seats, preventing duplicate invigilator assignments, and supporting both mixed-section and single-section seating modes. Generated seating plans, attendance sheets, and hall tickets are exported as HTML reports for printing. The application also provides a campus map, subject template library, and a dark/light theme toggle for enhanced usability. With its user-friendly interface and efficient data handling capabilities, the Exam Seating System significantly reduces administrative workload and improves the accuracy, consistency, and speed of university examination management.

LIST OF FIGURES

Figure No.	Figure Title	Page No.
1	Login Screen of Exam Seating System	15
2	Admin Dashboard – Room Layout Configuration	15
3	Student Data Import Screen	16
4	Seating & Schedule Generation Screen	16
5	Scheduled Exams List Screen	17
6	Exam Timetable Editor Screen	17
7	Hall Ticket Generation Screen	18
8	Invigilator Duty Roster Screen	18
9	Campus Map Screen	19
10	Search and Print Screen	19

LIST OF TABLES

Table no.	Table Title	Page no.
Table No.	Table Title	Page No.
1	Rooms Relational Database Table	9
2	Students Relational Database Table	9
3	Seating Plan Relational Database Table	10
4	Faculty Relational Database Table	10
5	Users Relational Database Table	10
6	Timetable Entries Relational Database Table	11
7	Room Broken Seats Relational Database Table	11
8	Exam Attendance Relational Database Table	11

ACRONYMS

- ESS : Exam Seating System
- JDBC : Java Database Connectivity
- GUI : Graphical User Interface
- SQL : Structured Query Language
- JVM : Java Virtual Machine
- DB : Database
- CSV : Comma Separated Values
- HTML : HyperText Markup Language
- IDE : Integrated Development Environment
- FN : Forenoon (Morning exam slot)
- AN : Afternoon (Evening exam slot)
- HOD : Head of the Department
- CRUD : Create, Read, Update, Delete

1. INTRODUCTION

1.1 Brief Information about the Project

The Exam Seating System (ESS) is a desktop-based application developed to automate and manage the examination seating process at Aditya University. The system provides a centralized platform for managing exam rooms, student records, faculty invigilators, seating plans, timetables, and hall tickets. It replaces traditional manual methods of seating arrangement with a fully computerized solution, thereby eliminating human errors, reducing preparation time, and ensuring a fair and systematic distribution of students across examination halls.

The application is developed using Core Java for the application logic, Java Swing for the graphical user interface, JDBC for database connectivity, and MySQL as the backend relational database. It supports two user roles — Administrator and Examiner — each with a dedicated set of features accessible through a sidebar navigation system. The application is designed to be intuitive, efficient, and suitable for use in large university examination departments.

1.2 Motivation and Contribution of Project

In most universities, examination seating is still managed manually — faculty members manually prepare seating charts, write student names on boards, and distribute hall tickets by hand. This approach is time-consuming, error-prone, and difficult to scale as student strength increases. Common problems include duplicate seat assignments, improper section mixing, unassigned invigilators, and loss of paper-based records.

These challenges motivated the development of the Exam Seating System. The key contributions of this project include:

- Automating seat allocation using a column-wise filling algorithm that distributes students efficiently across available rooms
- Supporting both mixed-section mode (students from different sections seated together) and section-wise mode
- Automatically assigning invigilators to rooms without duplication for the same exam slot
- Allowing the Administrator to mark broken or damaged seats, which are then automatically skipped during allocation
- Generating hall tickets, seating charts, and invigilator roster reports as printable HTML documents
- Providing a persistent exam timetable with Forenoon (FN) and Afternoon (AN) slot support for multiple branches
- Enabling backup and restoration of the complete database for data safety

1.3 Objectives of the Project

The main objectives of the Exam Seating System are:

- To develop a role-based desktop application for managing examination seating in a university
- To automate the process of allocating students to exam halls based on sections, room capacity, and seat availability
- To store and manage student records, room configurations, faculty data, and exam timetables in a structured MySQL database
- To generate hall tickets with student details, subject name, exam date, time, and seat number
- To maintain an invigilator duty roster and enable export of seating plans and attendance sheets as HTML reports
- To provide a user-friendly graphical interface with light and dark theme support for better usability
- To support CSV-based bulk import of student data for quick onboarding of large batches

1.4 Scope of the Project

The Exam Seating System can be deployed in universities, colleges, and academic institutions to manage the complete examination seating lifecycle. The system handles the following operations:

- Configuring exam halls with row-column layout, building (bhavan) assignment, and broken seat marking
- Importing student data in bulk from CSV files and managing records
- Generating seating schedules for one or more sections across multiple exam halls for a given subject, date, and time slot
- Managing the exam timetable for multiple branches with Forenoon and Afternoon sessions
- Creating and printing hall tickets for individual students or entire batches
- Maintaining a faculty invigilator roster and exporting it for official use
- Tracking exam attendance by marking students as Present or Absent against the seating plan
- Providing administrative functions such as user management, database backup and restoration, and subject/template library management

The system currently operates as a standalone desktop application connected to a local MySQL database, and is designed to be scalable for future web-based deployment.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations that the Exam Seating System must perform to effectively manage examination seating and related activities.

Room Layout Management

The system must allow the Administrator to add, edit, and delete exam room configurations including room number, number of rows, number of columns, seating capacity, and the campus building (bhavan) to which the room belongs. The Administrator must also be able to mark individual seats within a room as broken or non-functional, so that those seats are automatically skipped during seating allocation.

Student Data Management

The system must support bulk import of student data from CSV files, including fields such as roll number, student name, section, branch, and email. The Administrator must be able to view, add, edit, and delete individual student records. Students should be categorized by section to enable section-wise or mixed-mode seating allocation.

Seating Schedule Generation

The Examiner must be able to generate an automatic seating plan by selecting one or more student sections, a subject, an exam date, start time, end time, and optionally a specific building filter. The system must fill seats column-wise across available rooms, skipping broken seats, and must assign one invigilator per room without repetition for the same exam slot. The system must support two allocation modes: Mixed Mode (students from multiple sections seated together in roll number order) and Section-wise Mode (students grouped by section before filling rooms).

Exam Timetable Management

The system must allow the Examiner to create and manage an examination timetable for multiple branches. Each timetable entry must include a branch name, exam date, subject name, course code, and session slot (Forenoon or Afternoon). The timetable must be persisted in the database and loaded automatically on application startup.

Hall Ticket Generation

The system must generate hall tickets for students based on their seating plan entries. Each hall ticket must include the student's name, roll number, subject, exam date, start and end time, room number, seat row and column, and the invigilator name. Hall tickets must be exportable as printable HTML files.

Invigilator Roster Management

The system must display a duty roster of all faculty members assigned as invigilators, filterable by exam date. The roster must be exportable as an HTML report. Faculty records including name, department, email, and phone must be manageable through the application.

Exam Attendance Tracking

The system must allow invigilators or examiners to mark student attendance (Present/Absent) for each exam. Attendance records must be stored in the database linked to the seating plan entry, exam date, subject, and room.

User Management

The Administrator must be able to create, update, and delete user accounts. Each user must be assigned a role (Admin or Examiner) with a username and password. Role-based access control must ensure that only authorized users can access specific features.

Backup and Restore

The system must support export of the complete MySQL database as a SQL dump file and allow restoration of the database from a previously saved backup file. This feature ensures data safety and recovery.

2.2 Non-Functional Requirements

Performance

The system must respond quickly to all operations including seating plan generation, student search, and report export. Batch operations such as generating seating for hundreds of students must complete within a few seconds using efficient SQL batch inserts and indexed queries.

Security

Only authenticated users with valid credentials should be able to access the system. Role-based access control must restrict Admin-only operations from Examiner users. All database queries must use parameterized prepared statements to prevent SQL injection.

Usability

The system must provide a clean and modern graphical interface with clearly labeled controls, intuitive navigation through a sidebar, and helpful error messages. It must support both light and dark themes and provide keyboard shortcuts for frequently used functions.

Reliability

The system must handle database errors, import failures, and invalid user inputs gracefully without crashing. All critical operations such as seating plan generation must be wrapped in database transactions with rollback capability to prevent partial or corrupt data.

Scalability

The system must handle hundreds of students, multiple sections, and several exam halls simultaneously without significant performance degradation. The database schema must use proper indexing to maintain query speed as data volume increases.

2.3 Requirements Specification

2.3.1 Minimum Hardware Requirements

- Processor: Intel Core i3 or above (64-bit)
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 20 GB free space
- Monitor: Standard display (1024×768 or higher resolution recommended)
- Keyboard and Mouse

2.3.2 Software Requirements

- Operating System: Windows 7 / 10 / 11 (or macOS / Linux)
- Programming Language: Java (JDK 11 or above)
- IDE: Visual Studio Code / IntelliJ IDEA
- Database: MySQL Server 8.0 or above
- Database Connectivity: JDBC (MySQL Connector/J)
- Web Browser: Any modern browser (for viewing exported HTML reports)

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to Java

Java is a high-level, class-based, object-oriented programming language developed by Sun Microsystems (now Oracle). It is one of the most widely used programming languages in the world for developing desktop, web, enterprise, and mobile applications. Java follows the principle of "Write Once, Run Anywhere" (WORA), meaning that compiled Java bytecode can run on any platform that supports a Java Virtual Machine (JVM) without recompilation.

In the Exam Seating System, Java is used to implement the complete application logic. It handles operations such as room configuration, student data management, seating plan generation using the column-wise filling algorithm, timetable management, and hall ticket generation. Java's features such as object-oriented design, exception handling, multi-threading for background database operations, and strong memory management enable the development of a reliable and efficient examination management system.

3.2 Framework / Libraries

3.2.1 Introduction to Java Swing

Java Swing is a part of the Java Foundation Classes (JFC) that provides a rich set of GUI components for building desktop applications. Swing is platform-independent and offers a wide variety of components including JFrame, JPanel, JButton, JTextField, JPasswordField, JTable, JComboBox, JScrollPane, and custom-painted components, allowing developers to build highly interactive and visually appealing user interfaces.

In the Exam Seating System, Swing is used to design the complete user interface, including the login screen, sidebar navigation, room configuration panel, student data panel, seating schedule generator, timetable editor, hall ticket viewer, invigilator roster, campus map, and all other views. Custom Swing components such as ModernButton, ModernTextField, and ModernPasswordField are used to implement rounded, styled controls. The interface supports both light and dark themes achieved through dynamically applied color variables. A CardLayout-based panel system manages switching between different views within a single window.

3.3 Database Technology

The Exam Seating System uses a relational database for storing and managing all application data including room configurations, student records, seating plans, timetable entries, faculty information, and user accounts. A relational database ensures structured data storage, supports complex queries using SQL, enforces data integrity through constraints, and provides efficient indexed retrieval of records.

3.3.1 JDBC (Java Database Connectivity)

JDBC is a standard Java API that enables Java applications to connect to and interact with relational databases. It acts as a bridge between the Java application layer and the database layer, allowing the execution of SQL queries and the retrieval or modification of data.

In the Exam Seating System, JDBC is used to perform all database operations including establishing a connection to the MySQL database, executing CREATE TABLE statements during application startup, performing parameterized INSERT, UPDATE, DELETE, and SELECT operations using PreparedStatement to prevent SQL injection, executing batch inserts for seating plan generation to improve performance, and handling transactions with commit and rollback support to ensure data consistency during critical operations such as schedule generation.

3.3.2 MySQL Database

MySQL is an open-source relational database management system (RDBMS) that uses Structured Query Language (SQL) for data management. It is one of the most widely used databases for application development due to its high performance, reliability, ease of use, and strong community support.

In the Exam Seating System, MySQL is used to store all application data across eleven tables: rooms, students, seating_plan, faculty, users, subject_library, exam_templates, timetable_entries, exam_attendance, room_broken_seats, and room_status. MySQL provides features such as InnoDB storage engine for ACID-compliant transactions, foreign key constraints for referential integrity, indexed columns for fast query performance, and support for the ON DUPLICATE KEY UPDATE clause used in the timetable and subject library persistence logic.

3.4 Tools & IDEs

3.4.1 Visual Studio Code / IntelliJ IDEA

Visual Studio Code (VS Code) is a lightweight, open-source code editor developed by Microsoft that supports Java development through the Java Extension Pack. It provides features such as syntax highlighting, IntelliSense code completion, integrated terminal, debugger support, and Git integration, making it suitable for developing and testing Java applications efficiently.

IntelliJ IDEA is a full-featured Java IDE developed by JetBrains that provides advanced code analysis, refactoring tools, a powerful debugger, and built-in database tools. Both IDEs were used in the development of the Exam Seating System for writing, testing, and debugging the Java code.

4. DATABASE DESIGN

4.1 Database Tables

The Exam Seating System uses a relational database named projectdb running on MySQL. The database consists of eleven tables that together support all the application's data management requirements. These tables cover room information, student data, seating allocations, faculty details, user credentials, examination timetable, subject library, exam templates, attendance records, broken seat tracking, and room status management. The separation of data into normalized tables reduces redundancy and ensures data integrity through primary key, foreign key, and constraint definitions.

4.2 Table Structure

4.2.1 Rooms Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique room record ID	Primary Key
room_no	VARCHAR(20)	Room number/label	NOT NULL, UNIQUE
rows	INT	Number of seat rows	NOT NULL
cols	INT	Number of seat columns	NOT NULL
capacity	INT	Total seating capacity	NOT NULL
building	VARCHAR(100)	Campus bhavan name	DEFAULT NULL

Table 1: Rooms Relational Database Table

4.2.2 Students Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique student record ID	Primary Key
roll_no	VARCHAR(30)	Student roll number	NOT NULL, UNIQUE
name	VARCHAR(200)	Student full name	NOT NULL
section	VARCHAR(50)	Student section code	DEFAULT NULL
branch	VARCHAR(50)	Student branch/department	DEFAULT NULL
email	VARCHAR(200)	Student email address	DEFAULT NULL

Table 2: Students Relational Database Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique seating record ID	Primary Key
room_no	VARCHAR(20)	Assigned room number	NOT NULL
student_roll	VARCHAR(30)	Student roll number	NOT NULL
student_name	VARCHAR(200)	Student full name	NOT NULL
subject	VARCHAR(200)	Exam subject name	NOT NULL
seat_row	INT	Allocated seat row index	NOT NULL
seat_col	INT	Allocated seat column index	NOT NULL
exam_date	DATE	Date of the exam	NOT NULL
start_time	VARCHAR(15)	Exam start time (HH:MM 24h)	NOT NULL
end_time	VARCHAR(15)	Exam end time (HH:MM 24h)	NOT NULL

invigilator_name	VARCHAR(200)	Assigned invigilator name	DEFAULT "
section	VARCHAR(50)	Student section code	DEFAULT "

4.2.3 Seating Plan Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique faculty record ID	Primary Key
name	VARCHAR(200)	Faculty full name	NOT NULL, UNIQUE
department	VARCHAR(100)	Department name	DEFAULT NULL
email	VARCHAR(200)	Faculty email address	DEFAULT NULL
phone	VARCHAR(20)	Contact number	DEFAULT NULL

Table 3: Seating Plan Relational Database Table

4.2.4 Faculty Table

Table 4: Faculty Relational Database Table

4.2.5 Users Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique user record ID	Primary Key
username	VARCHAR(100)	Login username	NOT NULL, UNIQUE
password	VARCHAR(200)	Login password	NOT NULL
role	VARCHAR(20)	User role (ADMIN / EXAMINER)	NOT NULL

Table 5: Users Relational Database Table

4.2.6 Timetable Entries Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique timetable entry ID	Primary Key
branch	VARCHAR(50)	Academic branch name	NOT NULL
exam_date	DATE	Date of the exam	NOT NULL
subject_name	VARCHAR(200)	Subject name	NOT NULL

course_code	VARCHAR(20)	Course code	DEFAULT ''
slot	VARCHAR(5)	Session slot (FN or AN)	DEFAULT 'FN'

Table 6: Timetable Entries Relational Database Table

4.2.7 Room Broken Seats Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique broken seat record ID	Primary Key
room_no	VARCHAR(20)	Room in which seat is broken	NOT NULL
row_idx	INT	Row index of the broken seat	NOT NULL
col_idx	INT	Column index of the broken seat	NOT NULL

Table 7: Room Broken Seats Relational Database Table

4.2.8 Exam Attendance Table

Field Name	Data Type	Description	Constraint
id	INT AUTO_INCREMENT	Unique attendance record ID	Primary Key
seating_id	INT	Reference to seating plan	NOT NULL

student_roll	VARCHAR(30)	Student roll number	NOT NULL
exam_date	DATE	Date of the exam	NOT NULL
subject	VARCHAR(200)	Exam subject	NOT NULL
room_no	VARCHAR(20)	Exam room number	NOT NULL
status	VARCHAR(10)	Attendance status	DEFAULT 'PRESENT'
marked_by	VARCHAR(100)	User who marked attendance	DEFAULT ''
marked_at	TIMESTAMP	Timestamp of marking	AUTO UPDATE

Table 8: Exam Attendance Relational Database Table

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate, consistent data storage throughout the system.

- Primary Keys: Every table has an AUTO_INCREMENT integer primary key (id) that uniquely identifies each record.
- Unique Keys: The room_no column in the rooms table, roll_no in the students table, username in the users table, and name in the faculty table are all declared as UNIQUE to prevent duplicate entries.
 - NOT NULL Constraints: Critical fields such as room_no, roll_no, student_roll, subject, exam_date, username, and password are declared NOT NULL to ensure mandatory data is always provided.
- DEFAULT Values: Non-mandatory fields such as building, email, branch, invigilator_name, section, and slot use DEFAULT values so that records can be inserted even when optional fields are not provided.
- Composite Unique Keys: The room_broken_seats table uses a composite unique key on (room_no, row_idx, col_idx) to prevent duplicate broken seat entries for the same position. The timetable_entries table uses a unique key on (branch, exam_date, slot) to prevent duplicate timetable entries.
- Indexes: Multiple indexed columns exist on seating_plan (exam_date, start_time, room_no, student_roll, subject) to enable fast query performance when generating reports and searching seating records.

4.4 Table Relationships

The tables in the Exam Seating System database are logically related as follows:

- The seating_plan table references room_no from the rooms table and student_roll from the students table, establishing a logical link between room allocation and student records.
 - The exam_attendance table references seating_id from the seating_plan table, linking each attendance record to a specific seat allocation.
- The room_broken_seats table references room_no from the rooms table, associating broken seat coordinates with a specific room.
- The timetable_entries table is independent but logically used alongside the seating_plan table to match exam date and slot information.
- The users table is independent and controls authentication and role-based access to all features of the system.

5. SYSTEM DESIGN

5.1 System Architecture

The Exam Seating System follows a three-layer architecture that separates presentation, application logic, and data management into distinct layers, ensuring better organization, maintainability, and security.

1. **Presentation Layer (User Interface)**

Developed using Java Swing. Provides all visual components including the login screen, sidebar navigation, room management panel, student data panel, seating schedule generator, timetable editor, hall ticket panel, invigilator roster, campus map, search panel, and settings. Custom-styled components (ModernButton, ModernTextField) are used throughout. Supports light and dark themes.

2. **Application Layer (Business Logic)**

Implements all core operations within the ExamSeatingSystem Java class. Key business logic components include the column-wise seating allocation algorithm (autoGenerateSchedule), time normalization utility (normalizeTime), section normalization, timetable persistence (saveTimetableToDB, loadTimetableFromDB), report generation (generateSeatingHTML, generateAttendanceHTML), hall ticket creation, and role-based sidebar management.

3. **Data Layer (Database)**

Managed using MySQL through JDBC. Stores and retrieves data from eleven database tables. All database initialization, schema creation, and migration are handled automatically on application startup through the initDatabase() method, ensuring the application can be deployed on any MySQL instance without manual schema setup.

5.2 Modules Description

The Exam Seating System is divided into the following major modules:

1. Login and Authentication Module

Handles user login with username and password validation against the MySQL users table. Detects the user role (ADMIN or EXAMINER) from the database and dynamically shows or hides sidebar navigation items based on the role. Supports background-threaded login to prevent UI freezing during database queries.

2. Room Management Module (Admin)

Allows the Administrator to add, edit, and delete exam room configurations including room number, rows, columns, capacity, and bhavan assignment. Provides a visual seat grid interface for marking specific seats as broken or non-functional. Displays room availability status (AVAILABLE/BUSY) for a given exam date and time slot.

3. Student Data Management Module (Admin)

Supports bulk import of student records from CSV files with column mapping support for various CSV formats. Allows manual addition, editing, and deletion of individual student records. Displays all students in a searchable and filterable table with section and branch information.

4. Seating Schedule Generation Module (Examiner)

The core module of the system. Implements the column-wise filling algorithm that distributes students across available exam rooms for a selected subject, date, time, and sections. Supports mixed-section mode and section-wise mode. Automatically assigns one invigilator per room. Skips broken seats during allocation. Generates seating charts and attendance sheets as printable HTML reports.

5. Exam Timetable Module (Examiner)

Provides a multi-branch exam timetable editor with date columns and branch rows. Supports both Forenoon (FN) and Afternoon (AN) session slots. Allows adding, editing, and deleting timetable entries with subject name and course code. Timetable data is persisted to the database and loaded automatically at startup.

6. Hall Ticket Module (Examiner)

Generates formatted hall tickets for individual students or all students of a selected section and exam date. Each hall ticket includes student details, subject, exam date and time, room number, seat position, and invigilator name. Hall tickets are exported as HTML files for printing.

7. Invigilator Roster Module

Displays a duty roster table of all invigilators assigned across exam slots, filterable by exam date. Supports real-time search by faculty name. Provides HTML export of the complete roster for official distribution.

8. Search and Print Module (Examiner)

Allows quick search of student seating information by roll number, name, or section. Displays the seat allocation details for the searched student including room number, row, column, invigilator, and exam time. Supports individual seating slip generation for printing.

9. User Management Module (Admin)

Allows the Administrator to create, update, and delete user accounts for both Admin and Examiner roles. Provides password reset functionality. Displays all users in a table with their roles.

10. Backup and Restore Module (Admin)

Enables export of the complete MySQL database as a SQL dump file for backup purposes. Supports restoration of the database from a previously saved SQL file. This module ensures data safety and supports disaster recovery.

6. SYSTEM IMPLEMENTATION

Login Screen:

Displays the login interface where the user enters username and password.

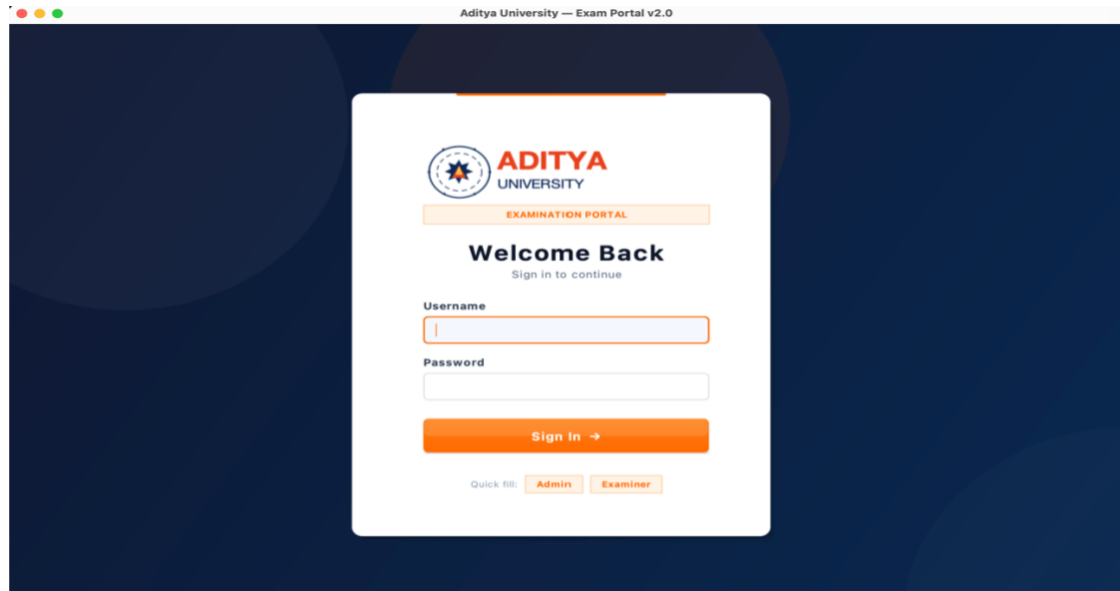


Figure 1: Login Screen of Exam Management System

Main Menu Screen:

Provides options such as Add Exam, Schedule Exam, Exam Timetable, Campus Map, Exam Invigilator, and Search & Print.

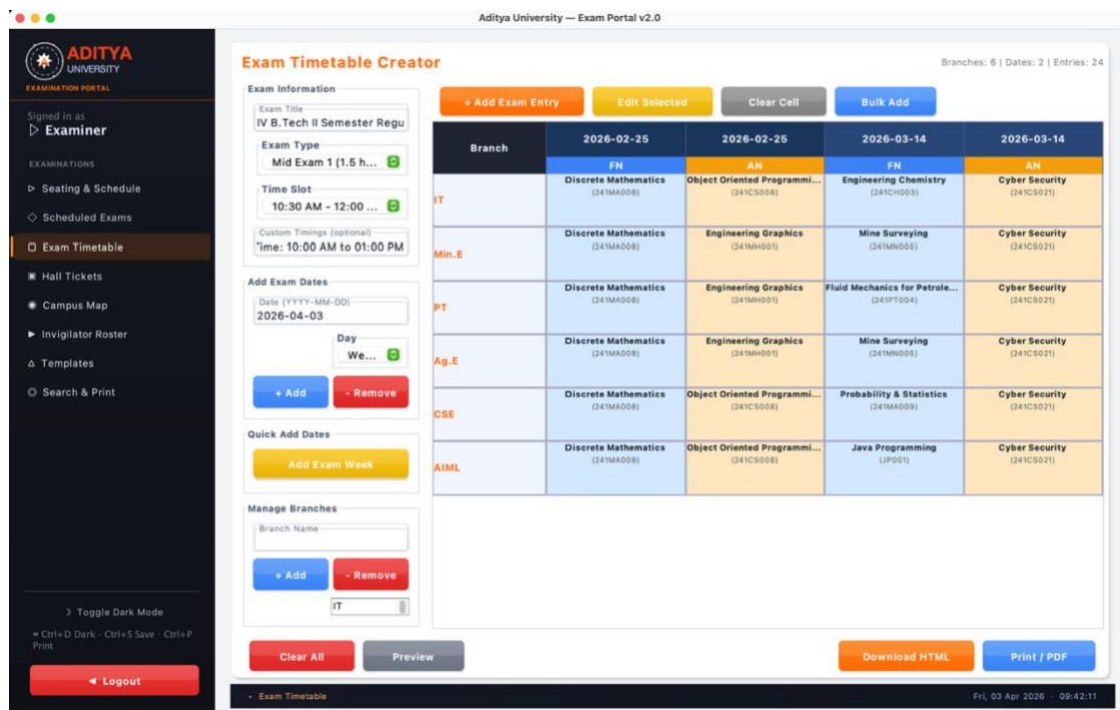


Figure 2: Main Menu Screen of Exam Management System

Create Schedule Screen:

Shows the form used to create an exam schedule based on the Timetable created.

Exam Scheduler

Auto-Fill from Timetable
2026-03-14 | FN | Engineering Chemistry (241CH003) | IT Refresh

Subject Details
Subject Name: Engineering Chemistry
Course Code: 241CH003
Exam Date (YYYY-MM-DD): 2026-03-14

Schedule Options
Session / Time Slot: AN (1:30 PM - 4:30 PM) Refresh

Allocation Mode
Roll No Wise Refresh

Preferred Bhavan / Building
Bill Gates Refresh

☒ Auto-assign Invigilators (Recommended)
Invigilator (or Auto): -- Auto-Assign -- Refresh

Check Rooms
Generate Schedule
View Halls

Select Sections (check/uncheck freely)
All None CSE AIN
5 sections selected
☒ CSE 1
☒ CSE 5
☒ CSE 7
☒ CSE 8
☒ CSE 9

Toggle Dark Mode
Ctrl+D Dark · Ctrl+S Save · Ctrl+P Print
Logout

Seating & Schedule Fri, 03 Apr 2026 09:44:38

Figure 3: Creating An Exam Schedule

Hall Ticket Generator Screen:

Displays the interface for Generating Hall Tickets for the Students.

Hall Ticket Generator

Find Student
24B11CS355
Search

Students
24B11CS355 - PRANAV VEDULA (CSE 9)

Generate Options
☒ Include Photo Placeholder
☒ Include QR Code
☒ Include Full Schedule
Generate Hall Tickets
Generate All (Sections)

Preview

ADITYA UNIVERSITY
EXAMINATION HALL TICKET
II B.Tech II Semester Regular Examinations - February/March 2026

Roll Number: 24B11CS355
Name: PRANAV VEDULA
Section: CSE 9
Branch: Computer Science & Engineering

Examination Schedule

Date	Subject	Time	Room	Seat
2026-02-23	ES	09:30 - 12:30	190	Row 20, Col 3
2026-02-23	Engineering Physics	09:30 - 12:30	112	Row 2, Col 2
2026-02-23	[FN] [FN] [FN] ES	09:30 - 12:30	116	Row 2, Col 3
2026-03-09	Discrete Mathematics	09:30 - 12:30	112	Row 1, Col 2
2026-03-14	Probability & Statistics	13:30 - 16:30	112	Row 1, Col 2

Toggle Dark Mode
Ctrl+D Dark · Ctrl+S Save · Ctrl+P Print
Logout

Hall Tickets Fri, 03 Apr 2026 09:44:02

Figure 4: Hall Ticket Generator of Exam Management System

Add,Modify,View,Edit Rooms Screen:{Admin Panel}

Displays all Room records in a table format.

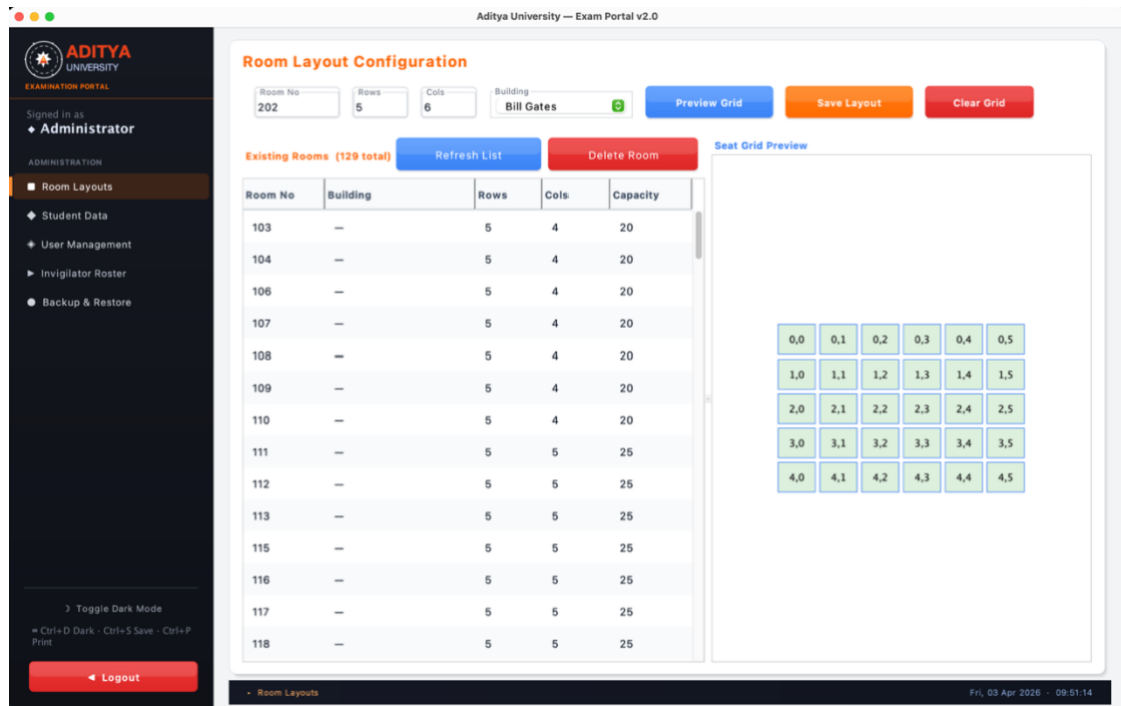


Figure 5: Rooms Information Screen of Exam Management System

Add Users, Invigilators and Management Screen

Shows the interface used to delete an employee record.

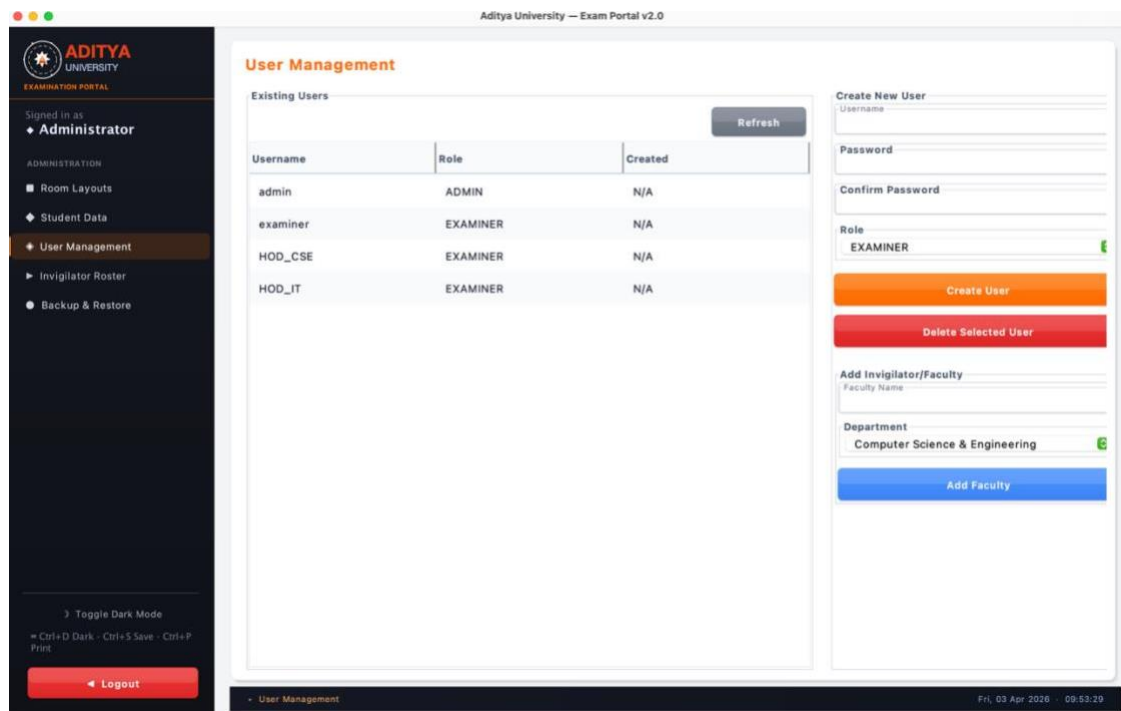


Figure 6: Employee Adding Screen of Exam Management System

7. CONCLUSION

The Exam Management System developed in this project provides basic functionalities for managing examination records. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.

Some possible future enhancements include:

- Online Examination Module:

Enabling students to take exams online with features like timer, auto-submit, and instant evaluation

- Result Management System:

Automating result calculation, grade generation, and performance analysis

- Student Portal:

Providing login access for students to view exam schedules, results, and notifications

- Web-Based Application:

Converting the desktop application into a web-based system for remote and anytime access

- Multi-User Support:

Allowing multiple users such as admins, faculty, and students to access the system simultaneously

- Improved User Interface:

Enhancing the design for a more interactive and user-friendly experience

These enhancements will make the system more scalable, efficient, and suitable for real-time academic environments.

9. REFERENCES

The following references were used during the development of the Exam Management System:

1. **Herbert Schildt**, *Java: The Complete Reference*, Oracle Press, 11th Edition.
2. **E. Balagurusamy**, *Programming with Java*, McGraw Hill Education.
3. **Korth, Silberschatz, Sudarshan**, *Database System Concepts*, McGraw Hill.
4. **Oracle Documentation**,
<https://docs.oracle.com>
5. **Java Documentation (Oracle)**,
<https://docs.oracle.com/javase>
6. **JDBC Tutorial – GeeksforGeeks**,
<https://www.geeksforgeeks.org/jdbc/>
7. **Java Swing Tutorial – TutorialsPoint**,
https://www.tutorialspoint.com/java_swing
8. **W3Schools Java Tutorial**,
<https://www.w3schools.com/java/>

10.APPENDIX

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the Java application and the Oracle database using JDBC.

DBConnection.java:

```
import java.sql.*;

public class DBConnection {
    public static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe","system","password");
        } catch (Exception e) {
            System.out.println(e);
        }
        return con;
    }
}
```

Insert Exam Record:

This code is used to add a new exam record into the database.

InsertExam.java:

```
import java.sql.*;

public class InsertExam {
    public static void main(String[] args) {
        try {
            Connection con = DBConnection.getConnection();
            String query = "INSERT INTO exam VALUES (?, ?, ?, ?, ?)";

            PreparedStatement pst = con.prepareStatement(query);
            pst.setInt(1, 101); // Exam ID
            pst.setString(2, "Data Structures"); // Subject
            pst.setString(3, "2026-04-10"); // Date
            pst.setString(4, "10:00 AM"); // Time
            pst.setInt(5, 100); // Marks
        }
    }
}
```

```

        int i = pst.executeUpdate();

        if (i > 0)
            System.out.println("Exam Added Successfully");

        con.close();
    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

A.2 SQL Queries

Create Student Table:

```

CREATE TABLE student (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(100) NOT NULL,
    course VARCHAR2(50),
    phone VARCHAR2(15),
    email VARCHAR2(100)
);

```

Create Exam Table:

```

CREATE TABLE exam (
    exam_id NUMBER PRIMARY KEY,
    subject VARCHAR2(100) NOT NULL,
    exam_date DATE,
    exam_time VARCHAR2(20),
    total_marks NUMBER
);

```

Create Result Table:

```

CREATE TABLE result (
    result_id NUMBER PRIMARY KEY,
    student_id NUMBER,
    exam_id NUMBER,
    marks_obtained NUMBER,
    grade VARCHAR2(5),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (exam_id) REFERENCES exam(exam_id)
);

```